



Desenvolvimento Web

Profª Sabrina B. Moreira



JavaScript

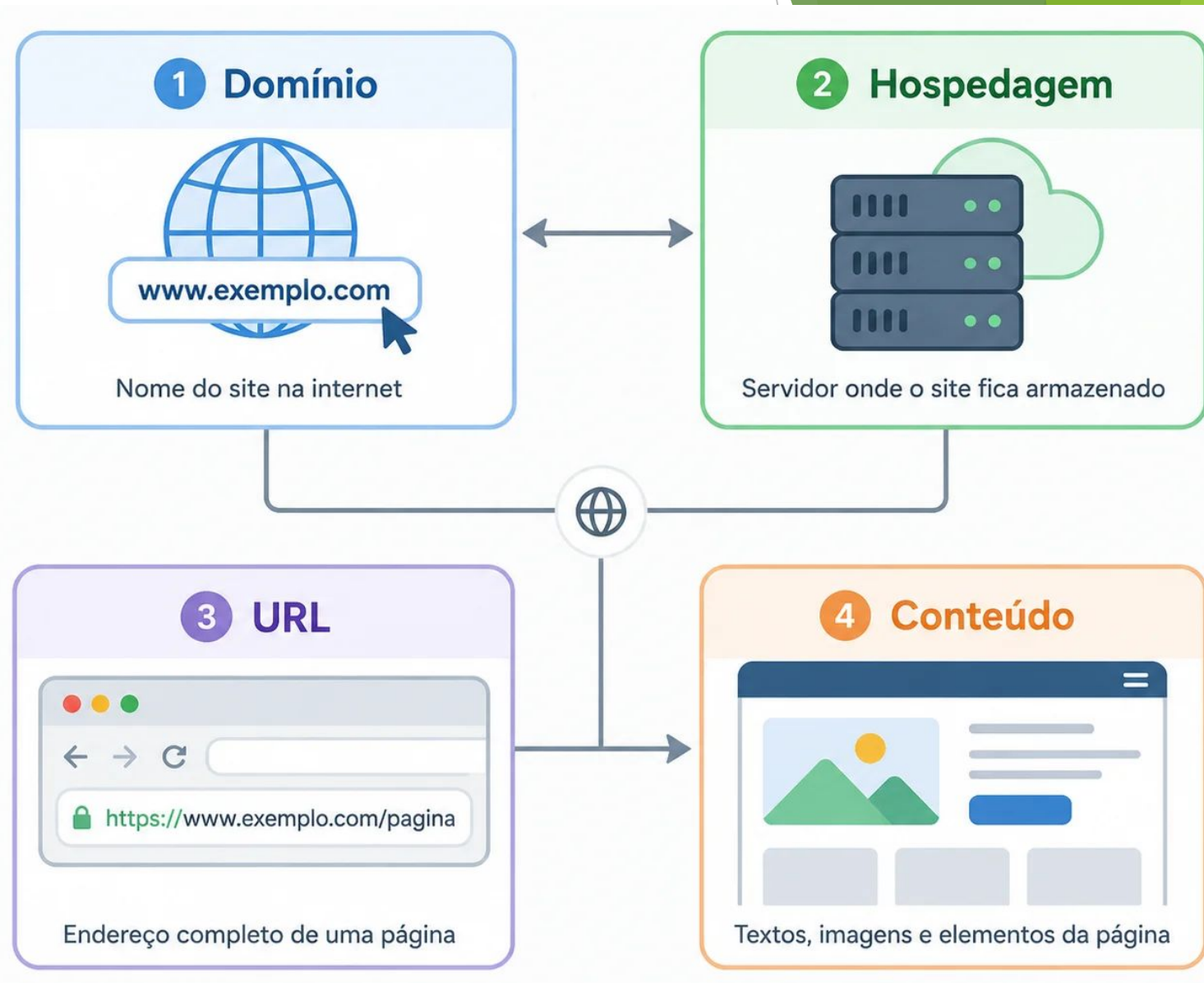
Aula 19.1: Conceitos web

The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern and dynamic visual effect. The word "Website" is centered in a clean, sans-serif font.

Website

O que é um website?

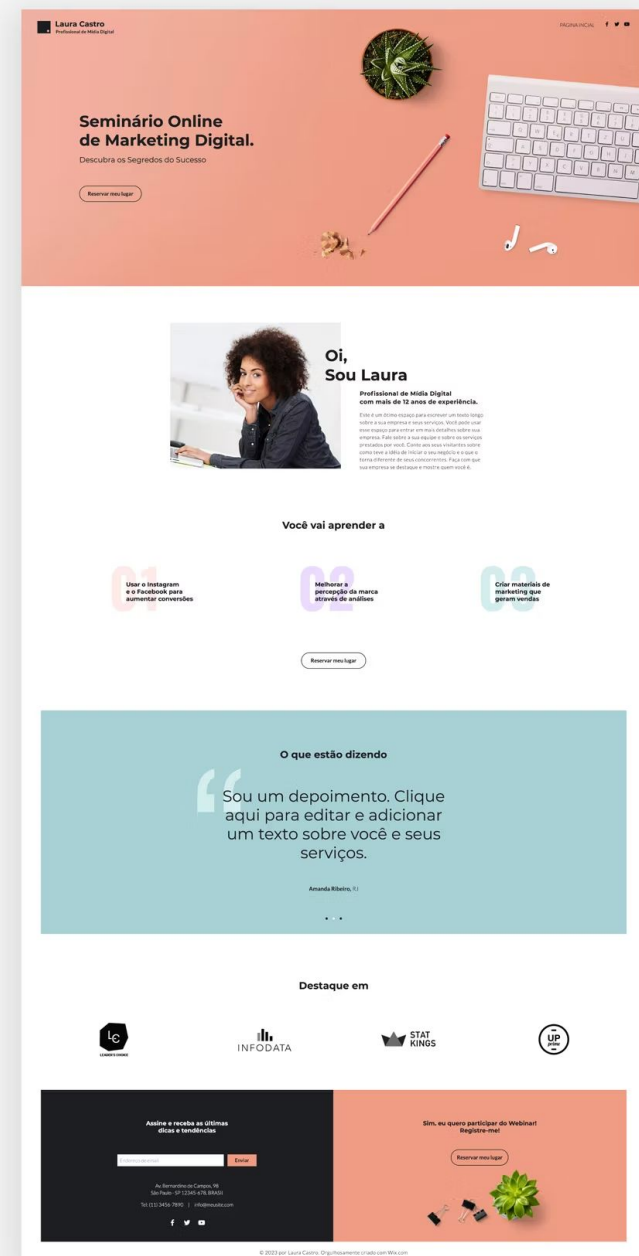
- ❖ Um website é um conjunto de páginas web interligadas e acessíveis por meio de um navegador, geralmente organizadas sob um mesmo domínio.
- ❖ Ele pode ter diferentes objetivos, como informar, vender, educar ou oferecer serviços.



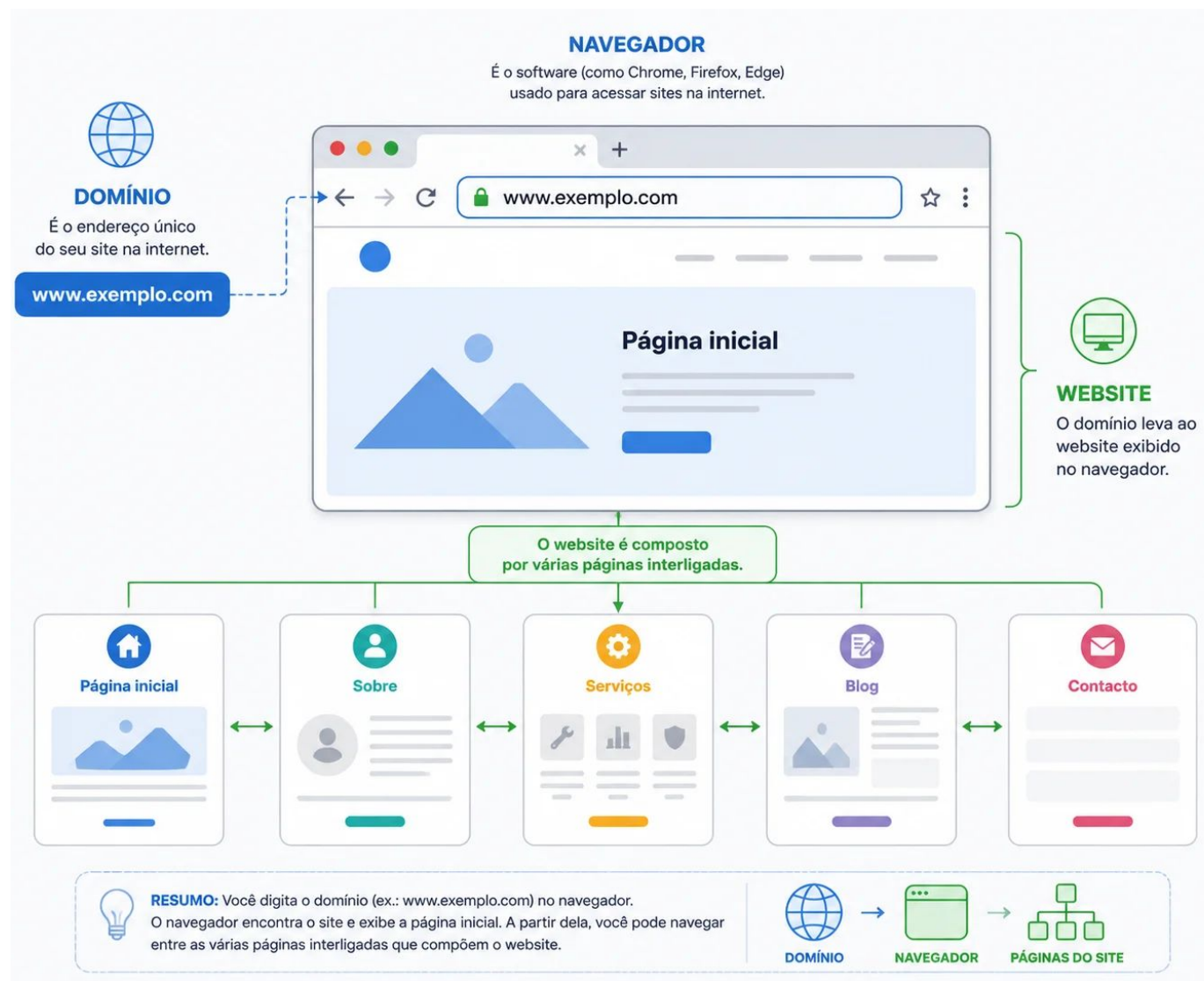
O que é um website?

Dentro desse contexto, é importante entender as diferenças entre alguns termos comuns:

- ❖ Uma página web é uma única página dentro de um website.
- ❖ Já a landing page é uma página web única criada com um objetivo específico e focado em conversão.
- ❖ Um sistema web é uma aplicação mais complexa acessada pelo navegador, geralmente com funcionalidades internas como login, cadastro, gestão de dados e interação com banco de dados.



O que é um website?



O que é um website?

- ❖ Enquanto uma aplicação web é um termo mais amplo que se refere a qualquer software que roda no navegador, podendo incluir tanto sistemas quanto ferramentas interativas (como editores online, redes sociais ou plataformas de streaming).



Fonte: webativa, 2026



JavaScript

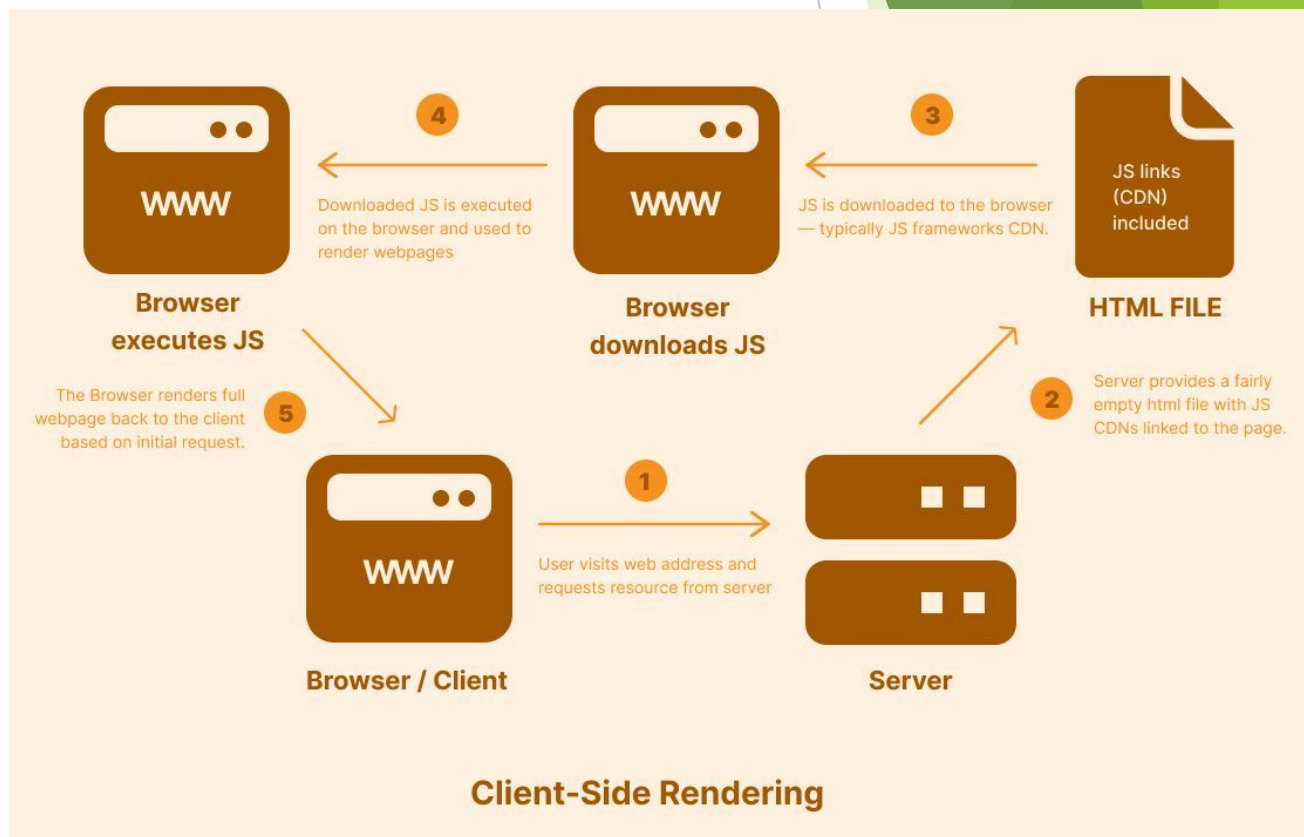
Aula 19.2: DOM, seletores e manipulação de elementos

HTML para objeto JS

O que acontece quando acessamos uma página?

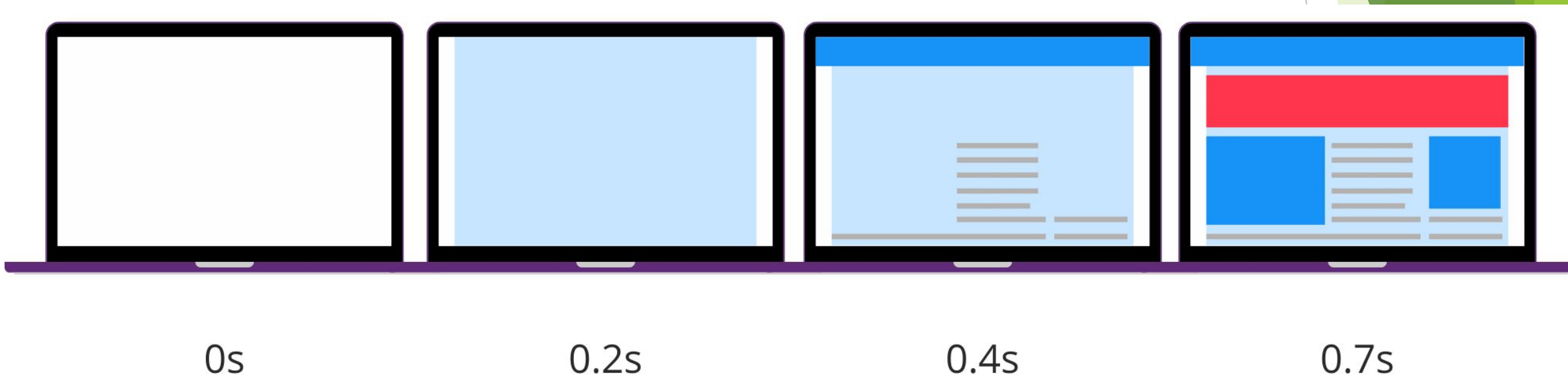
Quando digitamos um endereço no navegador:

- ❖ O navegador solicita o arquivo HTML ao servidor.
- ❖ O HTML é recebido em formato de texto.
- ❖ O navegador interpreta (faz o parsing) desse texto.
- ❖ É criada uma estrutura em memória chamada DOM.
- ❖ O JavaScript pode acessar e modificar essa estrutura.



O que acontece quando acessamos uma página?

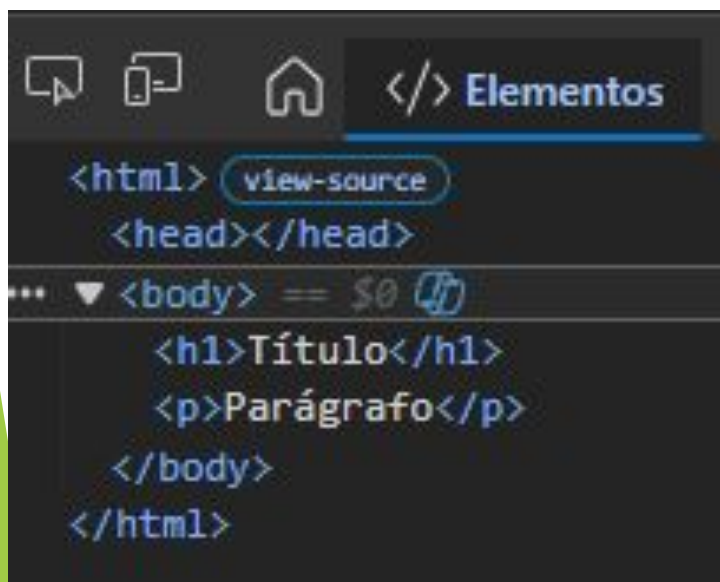
- ❖ O navegador não trabalha diretamente com o texto HTML, mas sim com objetos criados a partir dele.



Fonte: edgemesh, 2026

O HTML é apenas texto

- ❖ O código HTML originalmente é apenas texto.
- ❖ Para que a página seja **exibida e manipulada**, o navegador transforma esse texto em objetos.

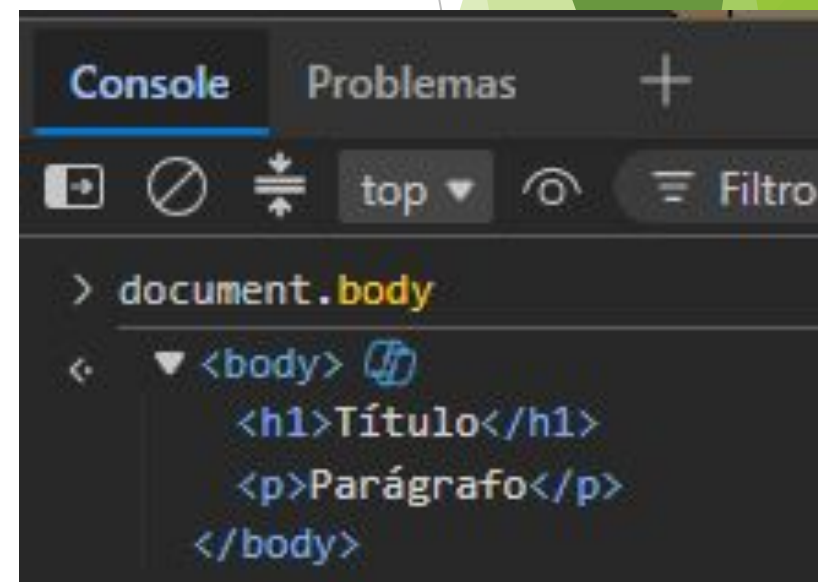


A screenshot of a web browser's developer tools, specifically the 'Elementos' (Elements) panel. The panel shows the HTML structure of a page. The root element is `<html>`, which contains a `<head>` and a `<body>`. Inside the `<body>`, there is an `<h1>Título</h1>` and a `<p>Parágrafo</p>`. A 'view-source' button is visible next to the `<html>` tag.

```
<html>
  <head></head>
  <body>
    <h1>Título</h1>
    <p>Parágrafo</p>
  </body>
</html>
```

Título

Parágrafo



A screenshot of a web browser's developer tools, specifically the 'Console' panel. The console shows the DOM tree structure. The root is `document.body`, which contains a `<body>` element. Inside the `<body>`, there is an `<h1>Título</h1>` and a `<p>Parágrafo</p>`.

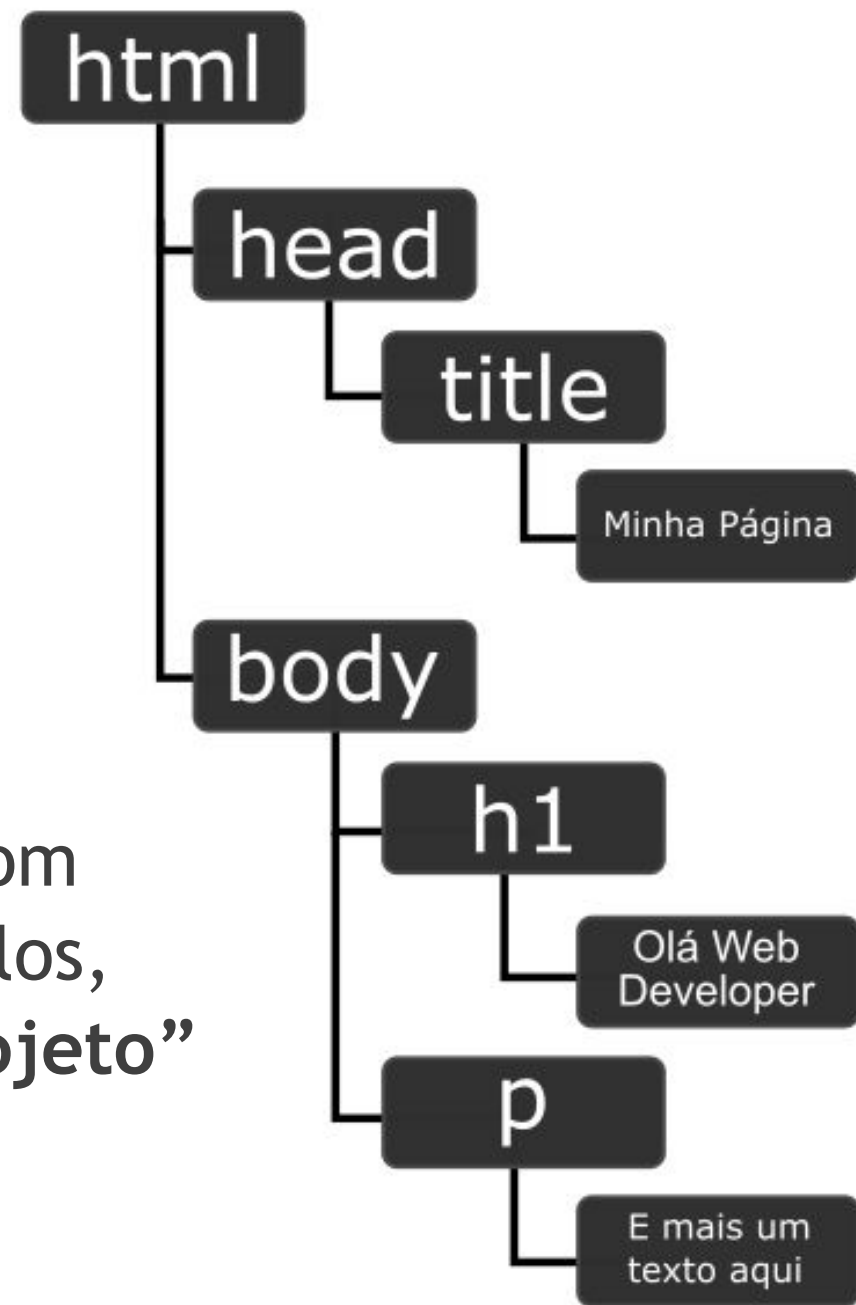
```
> document.body
  <body>
    <h1>Título</h1>
    <p>Parágrafo</p>
  </body>
```

The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern, layered effect. The shapes are concentrated on the left and right sides of the frame, leaving a large white central area.

DOM

DOM: Document Object Model

- ❖ É a forma como o navegador organiza e entende os elementos de uma página web.
- ❖ Quando um site é carregado, o HTML é transformado em uma estrutura parecida com uma árvore, onde cada elemento como títulos, textos, imagens, botões e links vira um “objeto” que pode ser acessado e alterado pelo JavaScript.



DOM: Document Object Model

- ❖ É graças ao DOM que uma página consegue **mudar conteúdo** sem precisar recarregar tudo, como atualizar mensagens, abrir menus, validar formulários ou alterar cores e estilos em tempo real.

Document
Object
Model

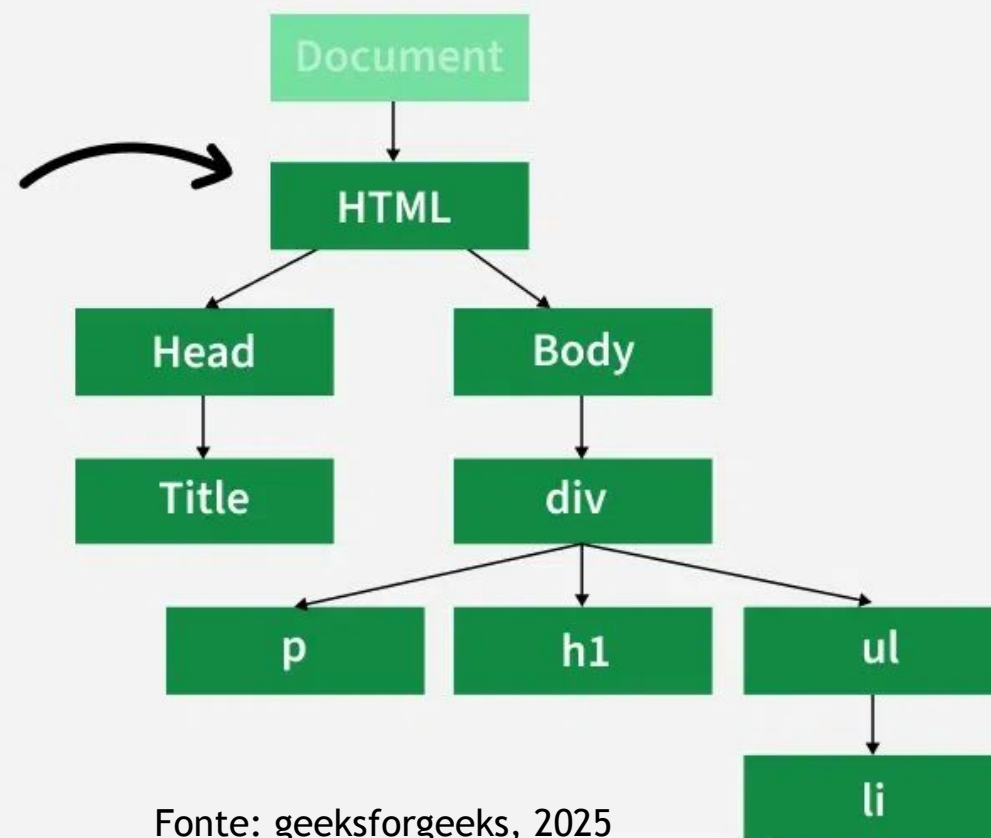
Fonte: programadorviking, 2025

DOM: Document Object Model

- ❖ O DOM funciona como uma **ponte** entre o código HTML da página e o JavaScript, permitindo que o site se torne interativo e dinâmico.

```
<html>
<head>
  <title> Webpage Title</title>
</head>
<body>
  <div>
    <h1>This is Heading Tag</h1>
    <p>Some Text Content</p>
    <ul>
      <li> List Item</li>
    </ul>
  </div>
</body>
</html>
```

The “DOM Tree”



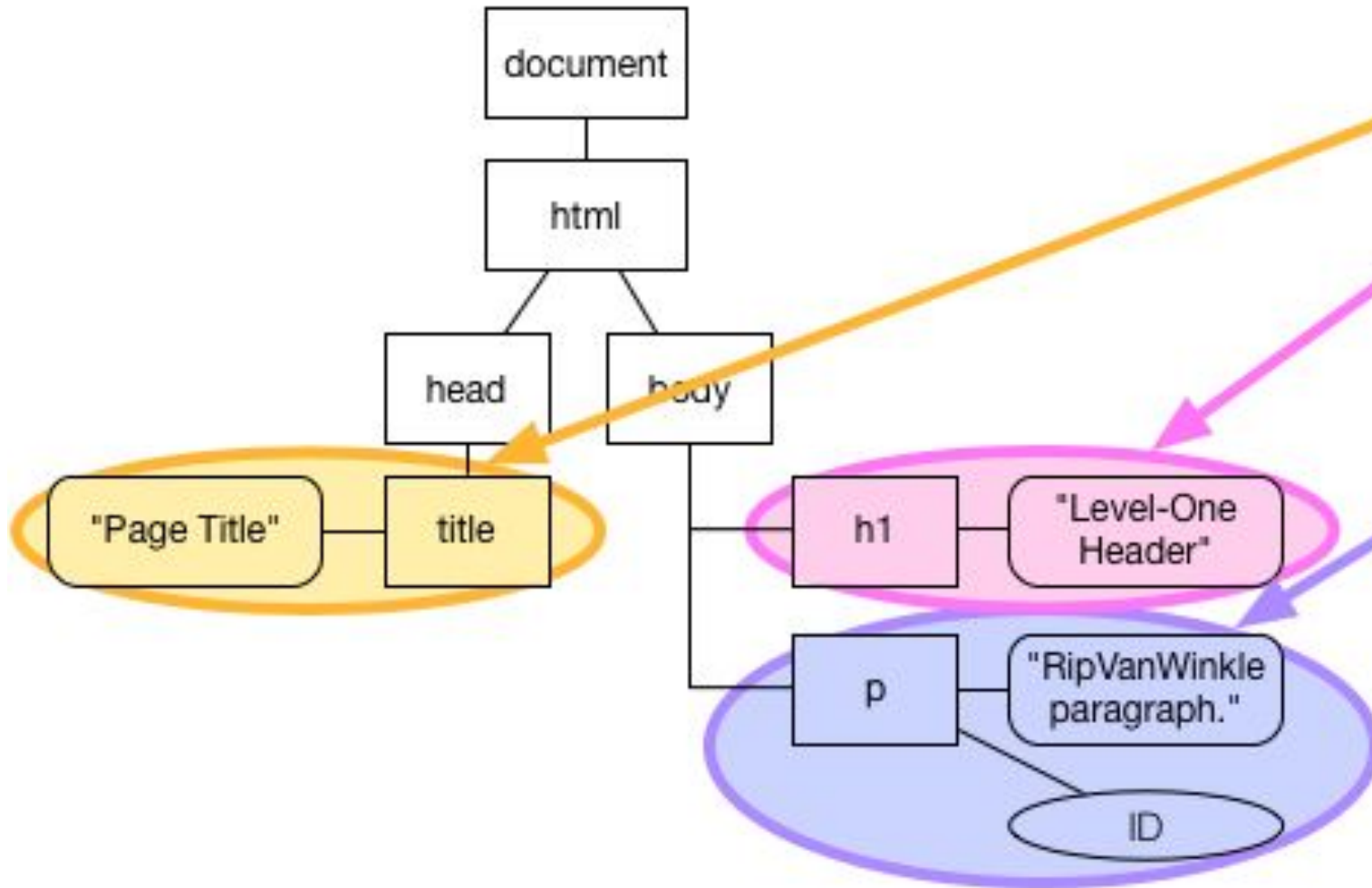
Fonte: geeksforgeeks, 2025

DOM: Document Object Model

- ❖ Ou seja, o DOM vê a página html como uma **árvore de elementos e atributos hierarquizados** que pode ser acessado por instruções JavaScript.
- ❖ Normalmente a conexão de HTML com JavaScript acontece por meio de eventos das tags html, como cliques, digitação, foco em elementos, etc.

DOM: Document Object Model

DOM tree for index.html



index.html

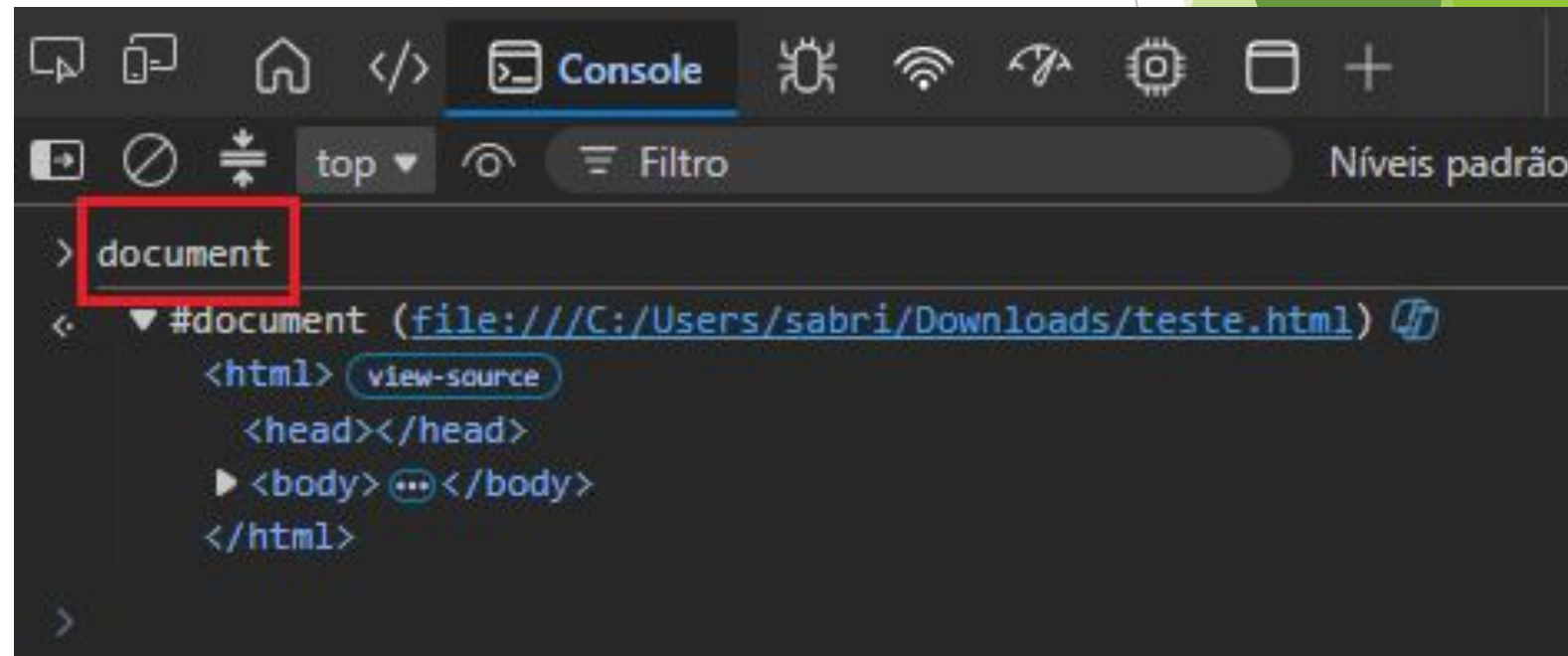
```
<html>
  <head>
    <title>Page Title</title>
  </head>
  <body>
    <h1>Level-One Header</h1>
    <p id="RipVanWinkle">
      RipVanWinkle paragraph.
    </p>
  </body>
</html>
```

DOM: Object Document

- ❖ O navegador disponibiliza o objeto global document que representa toda a página HTML.

Por meio dele podemos:

- ❖ localizar elementos
- ❖ criar novos elementos
- ❖ alterar conteúdos
- ❖ remover elementos



Seletor CSS

JS: Seletor CSS

- ❖ É uma regra ou padrão utilizado para identificar e selecionar elementos do documento HTML.
- ❖ Os principais tipos são os seletores de elemento, classe, ID, atributo, descendentes, filhos e as pseudoclasses.

```
<!-- Seletor de ID: #titulo -->
<h1 id="titulo">Curso de JavaScript</h1>

<!-- Seletor de elemento (tag): p -->
<p>Parágrafo introdutório.</p>

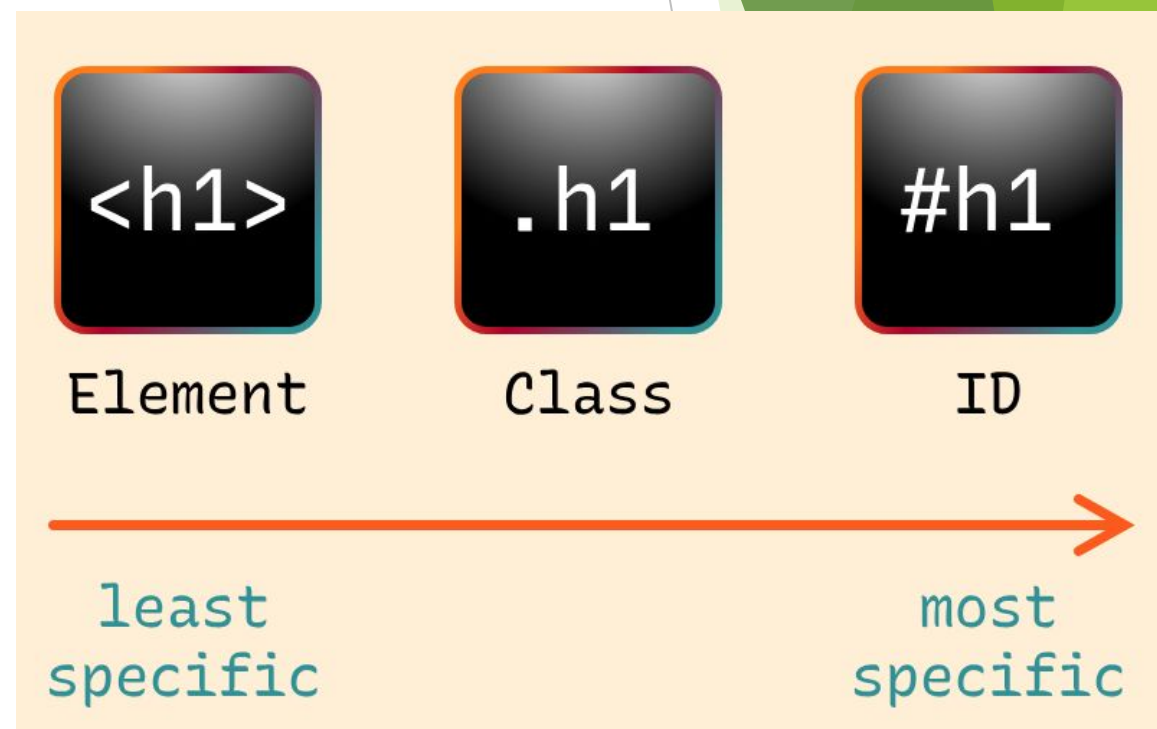
<!-- Seletor de classe: .item -->
<p class="item">Primeiro item.</p>
<p class="item">Segundo item.</p>

<!-- Seletor de atributo: [data-tipo] -->
<div class="card" data-tipo="produto">
| Card 1
</div>

<div class="card" data-tipo="produto">
| Card 2
</div>
```

JS: Seletor CSS

- ❖ No CSS, os seletores são utilizados para **aplicar estilos visuais** aos elementos da página, como cores, tamanhos e espaçamentos.
- ❖ Já no JavaScript, **métodos** utilizam como parâmetros esses seletores para **localizar e manipular** elementos dinamicamente.

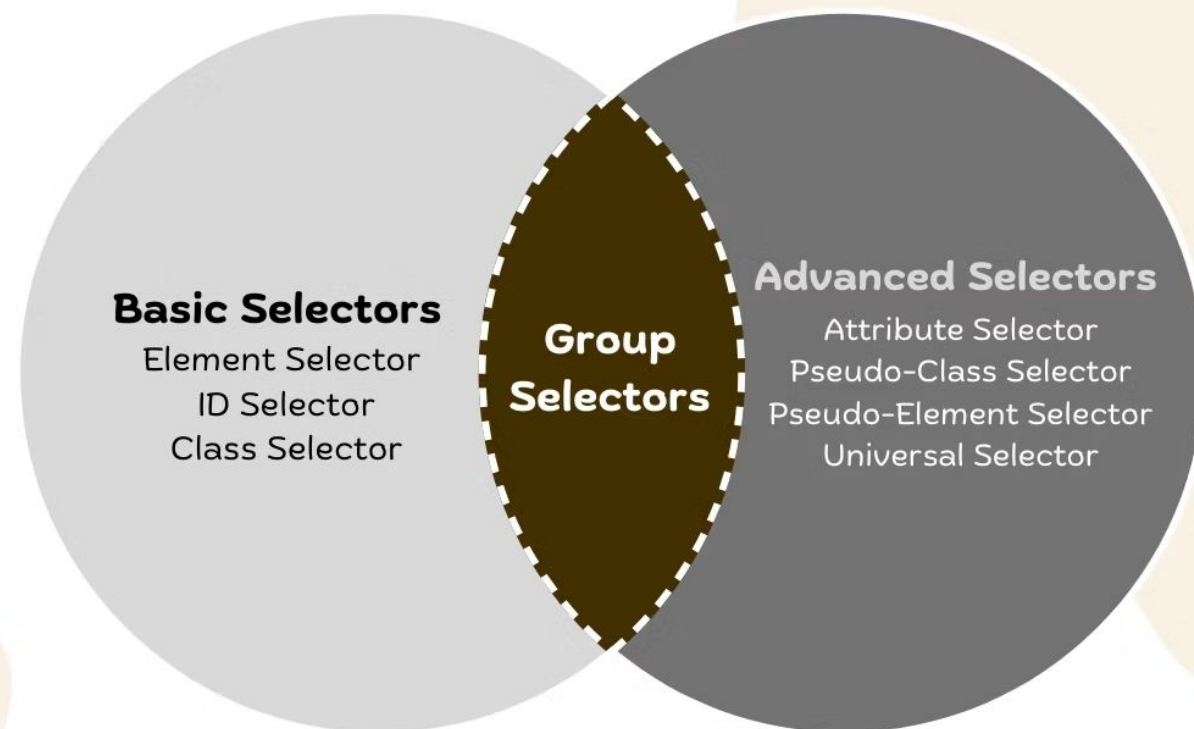


Fonte: Graham, 2024

JS: Seletor CSS

- ❖ Resumindo, os seletores CSS funcionam como uma linguagem comum entre **HTML, CSS e JavaScript**, permitindo identificar os mesmos elementos tanto para estilização quanto para manipulação de comportamento.

Types of CSS Selectors



JS: CSS Selectors

Artigo complementar.

- ❖ Clique na imagem abaixo para visitar o artigo complementar sobre o tema.



Manipulação de elementos no DOM

JS: Manipulação de elementos no DOM

- ❖ Em JavaScript, podemos manipular o DOM de forma dinâmica realizando captura de conteúdo, alteração, adição, remoção e até substituição de elementos e estilos CSS.
- ❖ Isso permite que a página deixe de ser estática e passe a responder às ações do usuário em **tempo real**.



JS: Manipulação de elementos no DOM

- ❖ A captura de conteúdo geralmente utiliza propriedades como *innerText* ou *textContent*, enquanto a alteração usa essas mesmas propriedades com novos valores.

```
<h1>
Hello, world!
</p>
This is a
paragraph.
</p>
```

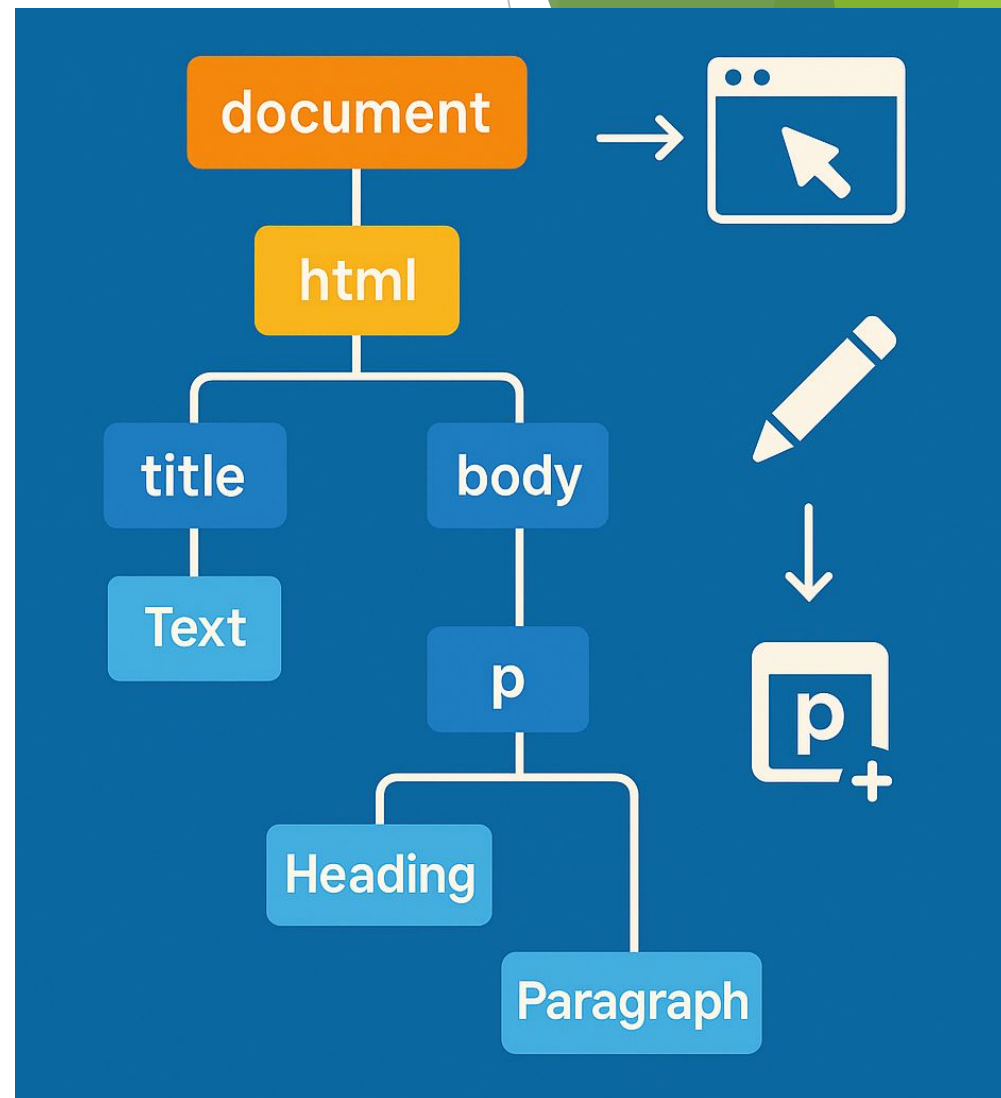
```
const element =
document.query-
selector("p")
element.textContent
="Hello, JavaScript!"
```

Change Text



JS: Manipulação de elementos no DOM

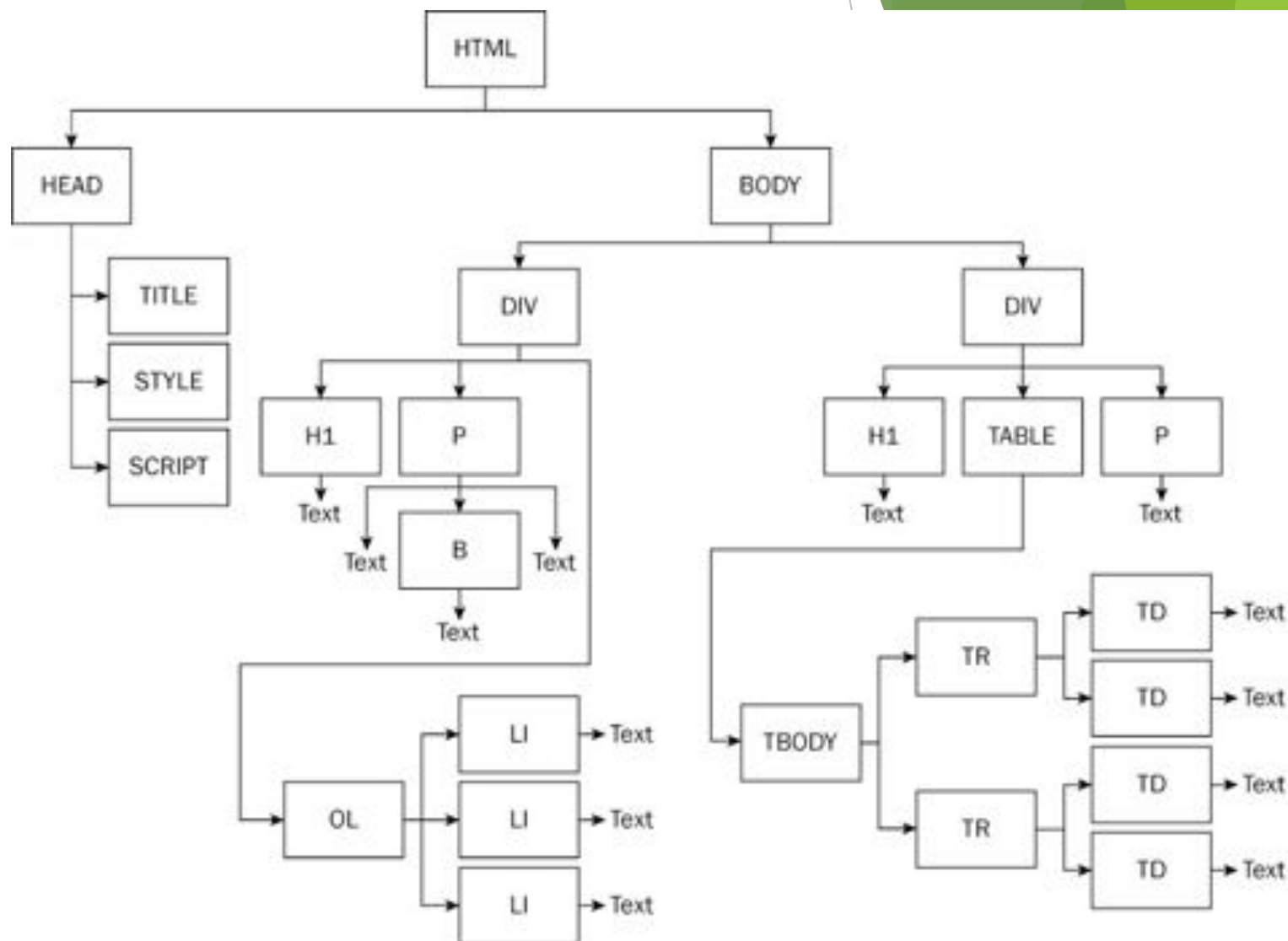
- ❖ Já a adição e remoção de elementos são feitas com métodos como *createElement()*, *appendChild()* e *remove()*, e a troca de elementos pode ser feita substituindo nós existentes no DOM.
- ❖ Além disso, também é possível alterar estilos diretamente via JavaScript através da propriedade *style* ou manipulando classes com *classList*.



Seleção de elementos no DOM

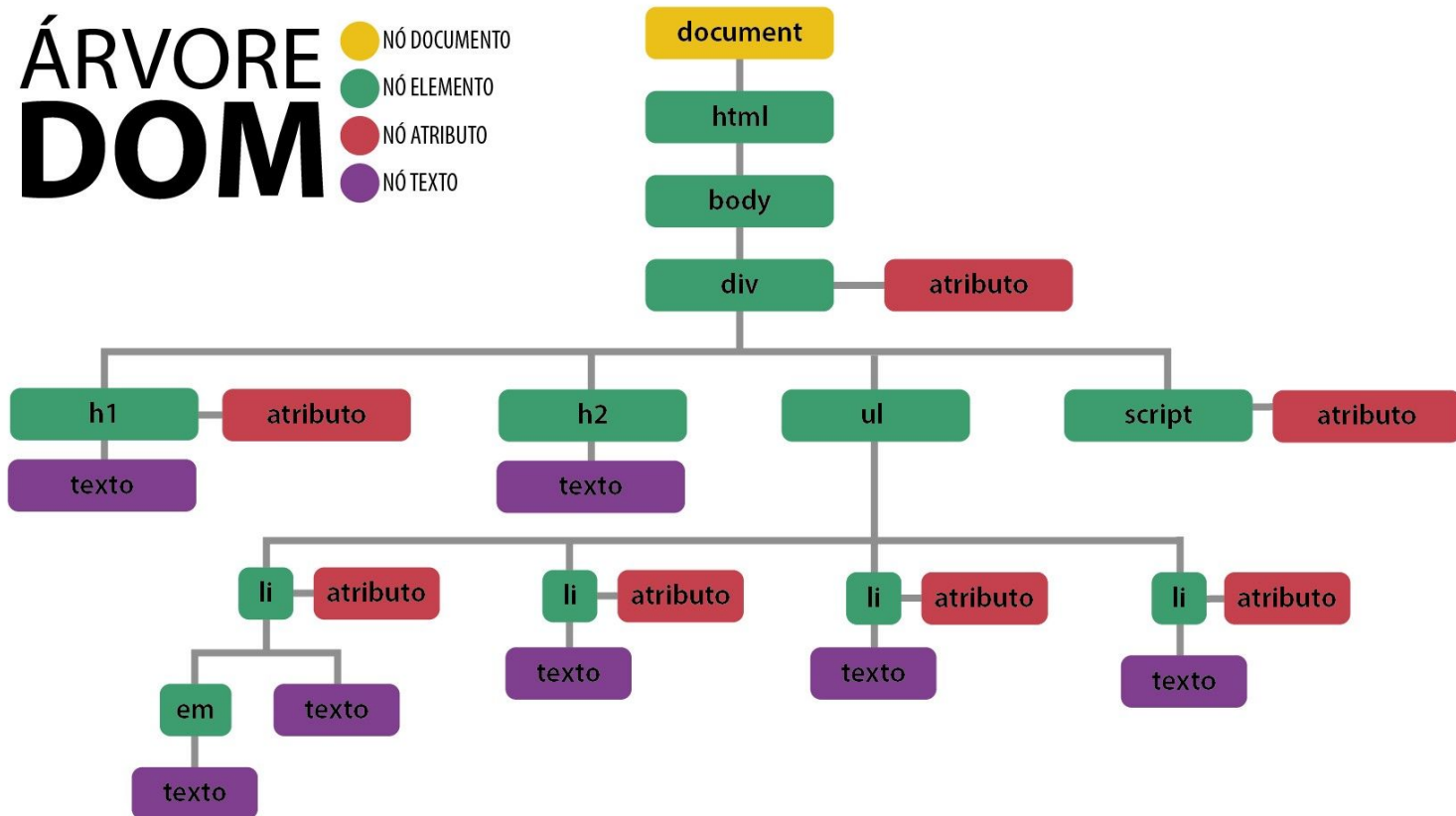
JS: Seleção de elementos no DOM

- ❖ Antes de manipular um elemento da página, o JavaScript precisa localizá-lo no DOM.



JS: Seleção de elementos no DOM

- ❖ Isso normalmente é feito utilizando métodos como `getElementById()` ou `querySelector()`, que permitem selecionar elementos por identificador, classe, tag ou outros seletores CSS.



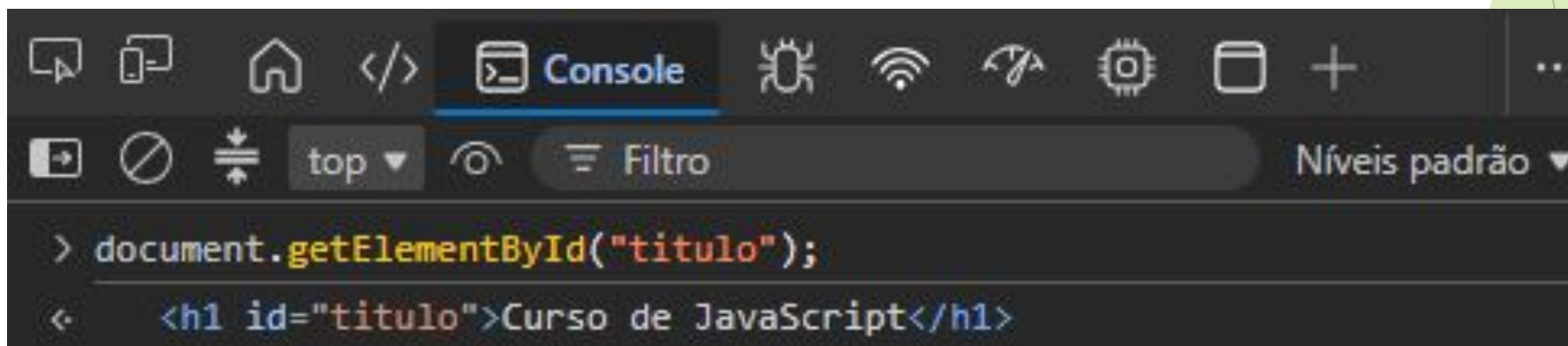
Fonte: Bruno, 2026

JS: Seleção de elementos no DOM

Método	Descrição	Flexibilidade	Performance
<code>document.getElementById()</code>	Seleciona um único elemento da página a partir do valor do atributo id.	Baixa, permite buscar apenas por id.	Muito alta pois realiza uma busca direta pelo identificador do elemento.
<code>document.getElementsByTagName()</code>	Seleciona todos os elementos que possuem uma determinada classe. Retorna uma coleção (HTMLCollection).	Baixa a média, permite buscar somente por classes CSS.	Alta, possui excelente desempenho e mantém uma coleção dinâmica atualizada automaticamente.
<code>document.querySelector()</code>	Seleciona o primeiro elemento que corresponde a um seletor informado.	Muito alta, aceita qualquer seletor válido.	Alta, ligeiramente mais lento que <code>getElementById()</code> , porém a diferença é insignificante na maioria dos casos.
<code>document.querySelectorAll()</code>	Seleciona todos os elementos que correspondem a um seletor. Retorna uma lista estática de nós (NodeList).	Muito alta, aceita qualquer seletor CSS válido.	Alta, pode ser um pouco mais custoso em seletores muito complexos ou grandes quantidades de elementos.

JS: getElementById()

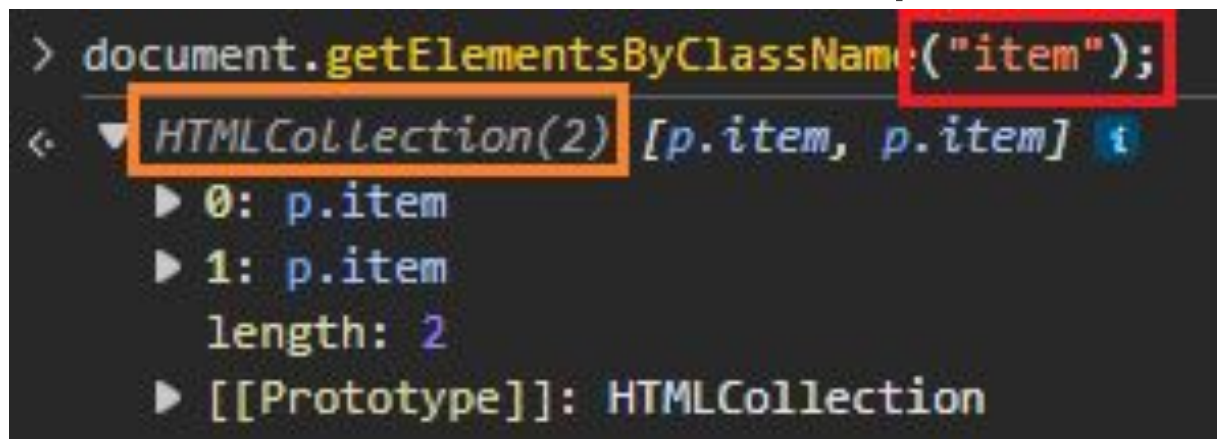
- ❖ É utilizado para selecionar um único elemento da página a partir do valor do atributo id.
- ❖ Como o **atributo id** deve ser único dentro de um documento HTML, esse método **sempre retorna apenas um elemento** ou null, caso nenhum elemento com o identificador informado seja encontrado.



```
> document.getElementById("titulo");  
< <h1 id="titulo">Curso de JavaScript</h1>
```

JS: `getElementsByClassName()`

- ❖ **Seleciona todos os elementos** que possuem uma determinada classe. Seu retorno é uma **coleção dinâmica** do tipo *HTMLCollection* (atualizada automaticamente).
- ❖ Esse método é útil quando vários elementos compartilham a mesma classe e precisam ser manipulados em conjunto, como itens de listas, cards ou componentes visuais semelhantes.



```
> document.getElementsByClassName("item");  
HTMLCollection(2) [p.item, p.item] i  
  ▶ 0: p.item  
  ▶ 1: p.item  
  length: 2  
  ▶ [[Prototype]]: HTMLCollection
```

JS: `querySelector()`

- ❖ Permite selecionar o primeiro elemento que corresponda a um seletor válido. Isso significa que ele pode buscar elementos por id, classe, nome da tag, atributos ou até **seletores mais complexos**.
- ❖ Sua flexibilidade faz com que seja um dos métodos mais utilizados no desenvolvimento com JavaScript.

```
> document.querySelector("p");  
< <p>Parágrafo introdutório.</p>
```

JS: querySelectorAll()

- ❖ Retorna todos os elementos que correspondem a um seletor informado. Esse método retorna uma coleção estática do tipo *NodeList*. Por ser estática, a coleção não é atualizada automaticamente caso ocorram alterações posteriores no DOM. Esse método suporta qualquer seletor válido, tornando-o extremamente versátil para selecionar grupos de elementos.

```
> document.querySelectorAll(".card");  
◀ ▼ NodeList(2) [div.card, div.card] ⓘ  
  ▶ 0: div.card  
  ▶ 1: div.card  
    length: 2  
  ▶ [[Prototype]]: NodeList
```

Tipos de coleções da DOM

JS: Seleção de elementos no DOM

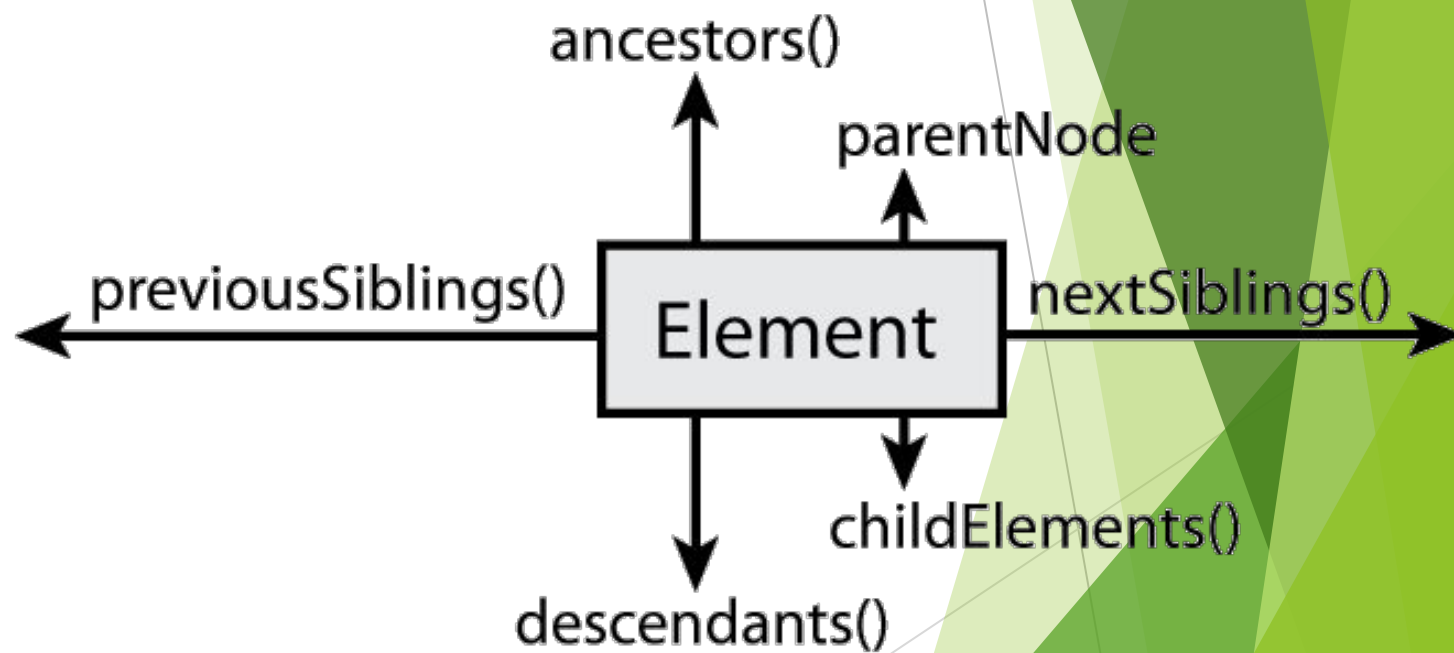
- ❖ A *HTMLCollection* é uma coleção dinâmica, que se atualiza automaticamente quando o DOM sofre alterações, como a adição ou remoção de elementos.
- ❖ Já a *NodeList*, é geralmente **estática**, mantendo o estado dos elementos no momento em que foi criada (com exceção de alguns métodos específicos como `childNodes`). Além disso, permite o uso de métodos como `forEach()`.

```
document.getElementsByClassName("item");  
▼ HTMLCollection(2) [p.item, p.item] ⓘ  
  ▶ 0: p.item  
  ▶ 1: p.item  
  length: 2  
  ▶ [[Prototype]]: HTMLCollection  
  
document.querySelectorAll(".card");  
▼ NodeList(2) [div.card, div.card] ⓘ  
  ▶ 0: div.card  
  ▶ 1: div.card  
  length: 2  
  ▶ [[Prototype]]: NodeList
```

Navegação por elementos

JS: Navegação pelos elementos do DOM

- ❖ No DOM (Document Object Model), além de **selecionar elementos diretamente**, também é possível navegar entre eles através das **relações de hierarquia**, como ancestrais, descendentes, filhos e irmãos.



Fonte: Levandoski, 2016

JS: Navegação

- ❖ Um elemento ancestral é aquele que está acima na hierarquia, enquanto um elemento descendente é qualquer elemento que está dentro de outro, independentemente do nível de profundidade.
- ❖ Já os filhos diretos são os elementos que estão imediatamente dentro de outro elemento.

```
document.querySelector("li").parentElement;
```

▼ 

- ▶ ...
- ▶ ...
- ▶ ...


```
document.querySelector("ul").querySelectorAll("li");
```

▼ NodeList(3) [li, li, li] 

- ▶ 0: li
- ▶ 1: li
- ▶ 2: li
- length: 3
- ▶ [[Prototype]]: NodeList

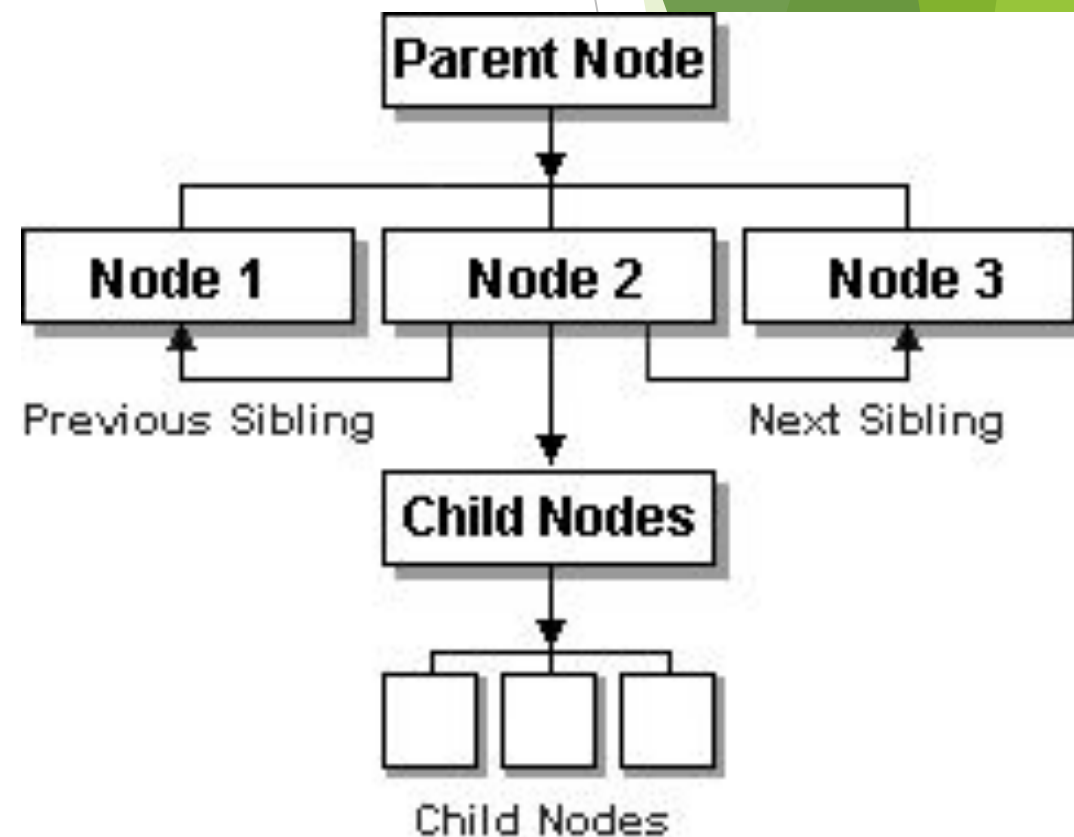
```
document.querySelector("ul").children;
```

▼ HTMLCollection(3) [li, li, li] 

- ▶ 0: li
- ▶ 1: li
- ▶ 2: li
- length: 3
- ▶ [[Prototype]]: HTMLCollection

JS: Navegação pelos elementos do DOM

- ❖ Os irmãos (*siblings*) são elementos que **compartilham o mesmo nível hierárquico**, podendo ser navegados como “anterior” ou “posterior”, como *nextElementSibling* (próximo irmão) e *previousElementSibling* (irmão anterior).
- ❖ Essa navegação permite **manipular a estrutura** da página de forma dinâmica sem necessariamente precisar de **novos seletores**.



JS: Navegação pelos elementos do DOM

Método	Descrição
Ascendente (Ancestral)	Elementos que estão acima na hierarquia do DOM. São os “pais”, “avós” e níveis superiores de um elemento.
Descendente	Todos os elementos que estão dentro de outro elemento, em qualquer nível de profundidade (filhos, netos, etc.).
Filho direto	Elementos que estão imediatamente dentro de outro elemento, ou seja, apenas o primeiro nível da hierarquia.
Pai (Parent)	Elemento diretamente acima de outro elemento na hierarquia. Cada elemento geralmente possui apenas um pai direto.
Irmão anterior (Previous Sibling)	Elemento que está no mesmo nível hierárquico e aparece imediatamente antes do elemento atual.
Irmão posterior (Next Sibling)	Elemento que está no mesmo nível hierárquico e aparece imediatamente depois do elemento atual.
Irmãos (Siblings)	Todos os elementos que compartilham o mesmo elemento pai, independentemente da posição.

JS: Entenda o que é o DOM no JavaScript

Artigo complementar.

- ❖ Clique na imagem abaixo para visitar o artigo complementar sobre o tema.



Captura e alteração de conteúdo

JS: textContent

- ❖ É uma propriedade que permite capturar ou alterar apenas o texto puro de um elemento, ignorando qualquer marcação HTML interna.
- ❖ Isso significa que ele não interpreta tags, retornando ou inserindo somente o conteúdo textual.

```
const title = document.getElementById("titulo");  
console.log(title.textContent);
```

Curso de JavaScript

JS: innerText

- ❖ Também é usado para **capturar ou alterar o texto visível** de um elemento, mas leva em consideração o estilo aplicado na página.
- ❖ Isso significa que ele retorna apenas o que está realmente visível ao usuário na tela, **ignorando elementos ocultos** por CSS e por isso, seu comportamento depende do estilo aplicado.

```
title.innerText = "Curso completo de front-end";  
console.log(title.innerText);  
  
Curso completo de front-end
```

JS: innerHTML

- ❖ Permite **capturar ou alterar o conteúdo completo** de um elemento, incluindo tags HTML internas. Diferente dos anteriores, ele interpreta HTML como estrutura, permitindo **inserir novos elementos** diretamente dentro de outro elemento.
- ❖ É muito poderoso e deve ser **usado com cuidado**, pois pode substituir completamente o conteúdo existente e também pode abrir brechas para problemas de segurança.

```
title.innerHTML = "<h1><strong>Curso completo de front-end.</strong></h1>";  
console.log(title.innerHTML);  
  
<h1><strong>Curso completo de front-end.</strong></h1>
```

JS: value

- ❖ É uma propriedade utilizada para **capturar ou alterar o valor de elementos** que possuem entrada de dados, como input, textarea e select.
- ❖ Diferente de textContent, innerText e innerHTML, que lidam com conteúdo exibido na tela, o value trabalha **diretamente com o valor interno de campos de formulário**, representando exatamente o que o usuário digitou ou selecionou.

```
const input = document.getElementById("nome");  
console.log(input.value);
```

Sabrina

```
input.value = "Sabrina B.";  
console.log(input.value);
```

Sabrina B.

Fonte: autora, 2026

JS: captura e alteração de conteúdo

Método	Descrição	Considera HTML?	Considera estilos?
textContent	Captura ou altera todo o texto do elemento, incluindo conteúdo oculto.	Não interpreta HTML (trata tudo como texto puro).	Não considera estilos visuais.
innerText	Captura ou altera apenas o texto visível na tela.	Não interpreta HTML.	Sim, considera o que está visível (display, visibility, etc).
innerHTML	Captura ou altera todo o conteúdo HTML interno do elemento.	Sim, interpreta e renderiza HTML.	Não depende de CSS.
value	Captura ou altera o valor de campos de entrada como input, textarea e select. Não funciona em qualquer elemento.	Não interpreta HTML.	Não depende de CSS.

JS: O que você precisa saber sobre o DOM

Artigo complementar.

- ❖ Clique na imagem abaixo para visitar o artigo complementar sobre o tema.



Fonte: Levandoski, 2026

Adição de conteúdo

JS: createElement()

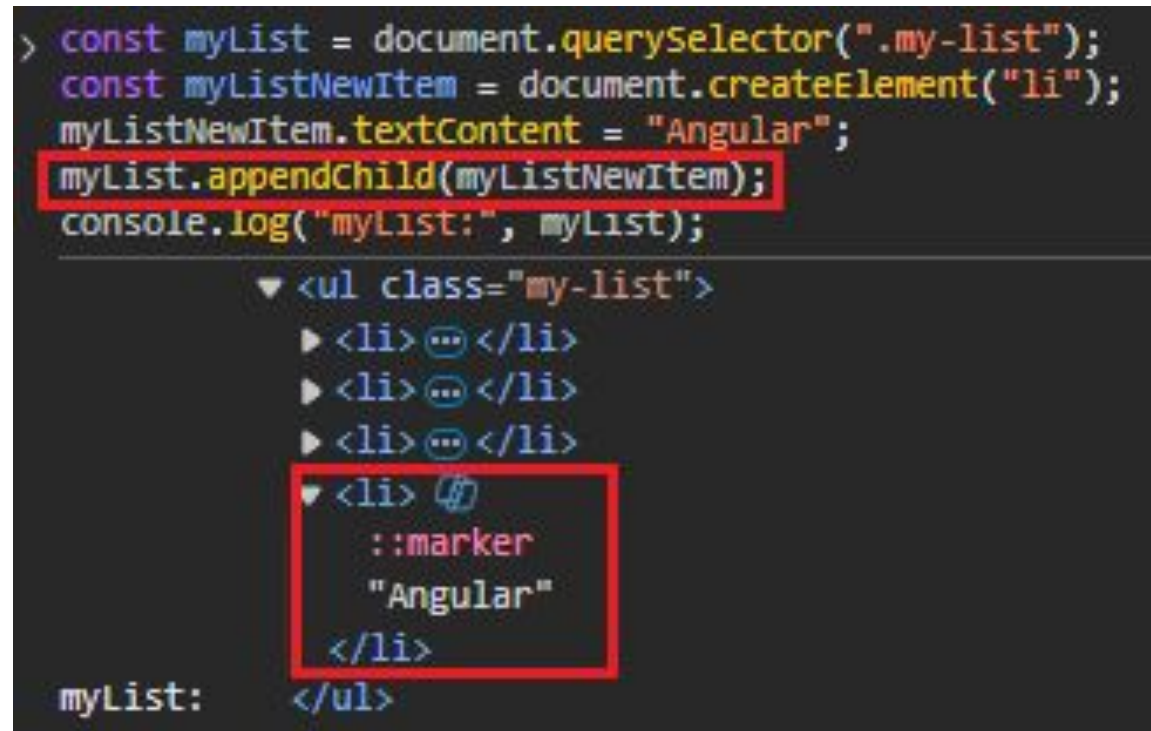
- ❖ Cria um novo elemento HTML em memória, mas ainda não o insere na página.
- ❖ É o primeiro passo para construir elementos dinamicamente, permitindo depois configurar seu conteúdo, classes e atributos antes de adicioná-lo ao DOM.

```
> const myList = document.querySelector(".my-list");  
const myListNewItem = document.createElement("li");  
myListNewItem.textContent = "Angular";  
console.log("myListNewItem:", myListNewItem);  
  
myListNewItem: <li>Angular</li>
```


JS: appendChild()

- ❖ Adiciona um elemento como último filho de um elemento pai. Ele é muito utilizado para **inserir** novos itens no final de listas ou estruturas já existentes.

```
> const myList = document.querySelector(".my-list");  
const myListNewItem = document.createElement("li");  
myListNewItem.textContent = "Angular";  
myList.appendChild(myListNewItem);  
console.log("myList:", myList);
```

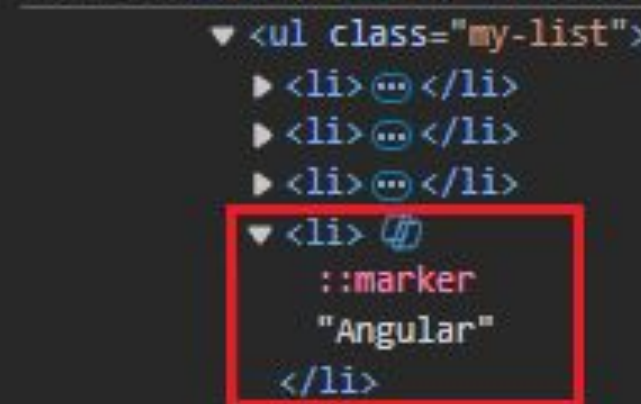


```
▼ <ul class="my-list">  
  ▶ <li>...</li>  
  ▶ <li>...</li>  
  ▶ <li>...</li>  
  ▼ <li>   
    ::marker  
    "Angular"  
  </li>  
myList: </ul>
```

JS: append()

- ❖ Adiciona elementos ao final de um elemento pai.
- ❖ Mas é mais moderno e flexível, pois permite inserir múltiplos elementos ou até texto diretamente.

```
> const myList = document.querySelector(".my-list");  
const myListNewItem = document.createElement("li");  
myListNewItem.textContent = "Angular";  
myList.append(myListNewItem);  
console.log("myList:", myList);
```

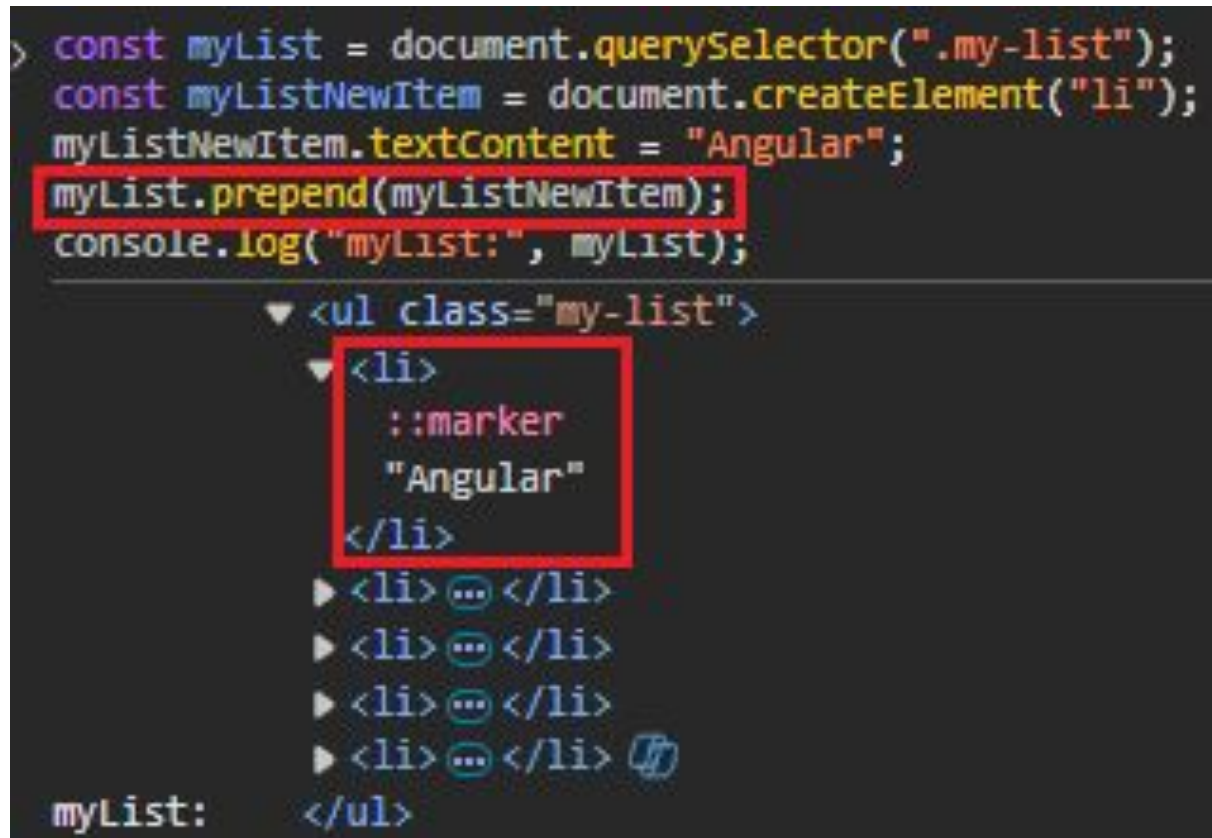


```
▼ <ul class="my-list">  
  ▶ <li> ... </li>  
  ▶ <li> ... </li>  
  ▶ <li> ... </li>  
  ▼ <li>   
    ::marker  
    "Angular"  
  </li>  
myList: <ul>
```

JS: prepend()

- ❖ Insere um elemento como **primeiro filho** de um elemento pai. Ou seja, no início da estrutura.

```
> const myList = document.querySelector(".my-list");  
const myListNewItem = document.createElement("li");  
myListNewItem.textContent = "Angular";  
myList.prepend(myListNewItem);  
console.log("myList:", myList);
```



The screenshot shows the DOM structure after the `prepend` operation. The `ul` element with class `my-list` now contains a new `li` element as its first child. The new `li` element has a `marker` attribute and the text content `"Angular"`. The console log shows the `myList` object, which is the `ul` element.

JS: insertBefore()

- ❖ Permite inserir um elemento antes de outro elemento específico dentro do mesmo pai.
- ❖ E por isso oferece maior controle sobre a posição da inserção.

```
> const myList = document.querySelector(".my-list");  
const myListNewItem = document.createElement("li");  
myListNewItem.textContent = "Angular";  
  
const referenceItem = myList.children[1];  
myList.insertBefore(myListNewItem, referenceItem);  
console.log("myList:", myList);
```

```
▼ <ul class="my-list">
```

```
▶ <li>⋮</li>
```

```
▼ <li>
```

```
  ::marker
```

```
  "Angular"
```

```
</li>
```

```
▶ <li>⋮</li> ⚙
```

```
▶ <li>⋮</li>
```

```
▶ <li>⋮</li>
```

```
myList:
```

```
</ul>
```

JS: métodos de adição de conteúdo

Método	Descrição	Uso
<code>createElement()</code>	Cria um novo elemento HTML em memória	Sempre que for construir novos elementos dinamicamente
<code>appendChild()</code>	Adiciona um elemento como último filho	Inserção simples no final da lista ou container
<code>append()</code>	Adiciona elementos ou texto no final	Quando precisa mais flexibilidade (múltiplos itens)
<code>prepend()</code>	Adiciona como primeiro filho	Quando o item deve aparecer no início
<code>insertBefore()</code>	Insere antes de um elemento específico	Quando a posição exata importa

Remoção e substituição de conteúdo

JS: removeChild()

- ❖ Remove um elemento filho a partir do elemento pai.
- ❖ Ele exige que o elemento seja referenciado pelo seu pai antes de ser removido.

```
> let MyNewList = document.querySelector(".my-list");
  console.log("MyNewList:", MyNewList);
  const MyNewListSecondItem = document.querySelectorAll(".my-list li")[1];
  console.log("Item que será removido:", MyNewListSecondItem);
```



The screenshot shows the browser's developer console with two states of the DOM tree. The top state shows a `<ul class="my-list">` containing three `<li>` elements: "HTML", "CSS", and "JavaScript". The second `<li>` ("CSS") is selected. Below the DOM tree, the console log shows "Item que será removido:" followed by the selected `<li>` element. The bottom state shows the same DOM tree after the `removeChild()` operation. The second `<li>` ("CSS") has been removed, leaving only "HTML" and "JavaScript". The line `MyNewList.removeChild(MyNewListSecondItem);` in the code is highlighted with a red box.

```
MyNewList: <ul class="my-list">
  <li>
    ::marker
    "HTML"
  </li>
  <li>
    ::marker
    "CSS"
  </li>
  <li>
    ::marker
    "JavaScript"
  </li>
</ul>

Item que será removido: <li>
  ::marker
  "CSS"
</li>

> MyNewList.removeChild(MyNewListSecondItem);
  console.log("MyNewList:", MyNewList);
```

```
MyNewList: <ul class="my-list">
  <li>
    ::marker
    "HTML"
  </li>
  <li>
    ::marker
    "JavaScript"
  </li>
</ul>
```


JS: remove()

- ❖ Remove diretamente o elemento selecionado do DOM.
- ❖ Ele é simples e moderno, sendo amplamente utilizado no JavaScript atual.

```
> let MyNewList = document.querySelector(".my-list");
  console.log("MyNewList:", MyNewList);
  <ul class="my-list">
    <li>
      ::marker
      "HTML"
    </li>
    <li>
      ::marker
      "CSS"
    </li>
    <li>
      ::marker
      "JavaScript"
    </li>
  </ul>
  MyNewList: <ul>
  <ul class="my-list">
    <li>
      ::marker
      "HTML"
    </li>
    <li>
      ::marker
      "JavaScript"
    </li>
  </ul>
  MyNewListSecondItem = document.querySelectorAll(".my-list li")[1];
  console.log("Item que será removido:", MyNewListSecondItem);
  MyNewListSecondItem.remove();
  Item que será removido: <li>CSS</li>
```

Fonte: autora, 2026

JS: replaceChild()

- ❖ Substitui um elemento filho por outro dentro de um elemento pai.
- ❖ Ele exige que tanto o elemento antigo quanto o novo sejam referenciados pelo elemento pai antes da substituição.

```
const lista = list.cloneNode(true);

const primeiroItemLista = lista.firstChild;

const novoItemLista = document.createElement("li");
novoItemLista.textContent = "CSS 3";

console.log("Lista (clone):", lista.innerHTML);
console.log("Filho que será substituído:", primeiroItemLista);

lista.replaceChild(novoItemLista, primeiroItemLista);

console.log("Lista atualizada (clone):", lista.innerHTML);
```

Lista (clone):

```
<li>CSS</li>
<li>JavaScript</li>
<li>TypeScript</li><li>React</li><li>Vue</li>
```

Filho que será substituído: CSS

Lista atualizada (clone):

```
<li>CSS 3</li>
<li>JavaScript</li>
<li>TypeScript</li><li>React</li><li>Vue</li>
```

JS: replaceChildren()

- ❖ Remove todos os **elementos filhos** atuais de um elemento e os substitui pelos **novos elementos** informados, sem a necessidade de remover cada filho individualmente.

```
> const novosFilhos = [];  
for (let i = 0; i <= 1; i++) {  
  const item = document.createElement("li");  
  item.textContent = "Filho adicionado dinamicamente";  
  novosFilhos.push(item);  
}  
const listaComFilhos = list.cloneNode(true);  
  
console.log("Lista:", listaComFilhos.innerHTML);  
  
listaComFilhos.replaceChildren(...newElements);  
console.log("Lista atualizada:", listaComFilhos.innerHTML);  
Lista:  
  
  <li>CSS</li>  
  <li>JavaScript</li>  
  <li>TypeScript</li><li>React</li><li>Vue</li>  
Lista atualizada: <li>Filho adicionado dinamicamente</li><li>Filho adicionado dinamicamente</li>
```

JS: remoção e substituição de conteúdo

Método	Descrição	Uso
<code>remove()</code>	Remove um elemento do DOM.	Quando for excluir permanentemente um elemento da página.
<code>removeChild()</code>	Remove um elemento filho de um elemento pai específico.	Quando for necessário remover um filho conhecido de um contêiner.
<code>replaceChild()</code>	Substitui um elemento filho por outro.	Quando um elemento deve ser removido e substituído por um novo.
<code>replaceChildren()</code>	Remove todos os filhos atuais e adiciona novos elementos.	Quando for necessário reconstruir completamente o conteúdo de um elemento.

Fonte: autora, 2026

Manipulação de estilos

JS: Manipulação de estilos

- ❖ A propriedade *style* altera **estilos inline** diretamente no elemento, sendo útil para mudanças rápidas.
- ❖ O *classList* é a forma mais recomendada, pois permite adicionar, remover ou alternar **classes CSS** sem misturar estrutura e apresentação.
- ❖ Já *className* substitui **todas as classes** do elemento de uma vez.

```
const element = document.querySelector('.myClass');  
element.style.color = 'blue';  
element.style.fontSize = '20px';
```

JS: Manipulação de estilos

- ❖ Os métodos *setAttribute()* e *removeAttribute()* manipulam **atributos HTML**, como **class** e **style**.
- ❖ Por fim, *getComputedStyle()* não altera estilos, apenas retorna o estilo final aplicado pelo navegador ao elemento.

```
const element = document.querySelector( '#myId' );  
element.classList.add( 'newClass' ); // Add a class  
element.classList.remove( 'oldClass' ); // Remove a class
```


JS: manipulação de estilo

Método	Descrição	Uso
<code>element.style</code>	Altera estilos CSS diretamente no elemento (estilo inline).	<code>card.style.backgroundColor = "blue";</code>
<code>element.classList.add()</code>	Adiciona uma ou mais classes CSS ao elemento.	<code>card.classList.add("ativo");</code>
<code>element.classList.remove()</code>	<code>element.classList.remove()</code>	<code>card.classList.remove("ativo");</code>
<code>element.classList.toggle()</code>	<code>element.classList.toggle()</code>	<code>card.classList.toggle("ativo");</code>
<code>element.classList.contains()</code>	<code>element.classList.contains()</code>	<code>card.classList.contains("ativo");</code>

JS: manipulação de estilo

Método	Descrição	Uso
<code>element.className</code>	Define ou retorna todas as classes do elemento em formato de string.	<code>card.className = "card destaque";</code>
<code>element.setAttribute()</code>	<code>card.setAttribute("class", "card ativo");</code>	<code>card.setAttribute("class", "card ativo");</code>
<code>card.setAttribute("class", "card ativo");</code>	Remove um atributo do elemento.	Remove um atributo do elemento.
<code>window.getComputedStyle()</code>	<code>card.removeAttribute("style");</code>	<code>window.getComputedStyle()</code>

Fonte: autora, 2026

The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern and dynamic visual effect.

Dúvidas?

Vídeos complementares

- ◆ Assista os vídeos abaixo para revisar o conteúdo dessa aula.



Exercício

Exercício

- ❖ Agora teste você em seu navegador os scripts vistos nessa aula.
- ❖ Acesse o **arquivo de exemplo** no link abaixo.
- ❖ Copie cada bloco de código de exemplo, uma a uma na aba console das **ferramentas de desenvolvedor** do seu navegador, e analise os resultados.
- ❖ Exemplos

Referência da aula

- ❖ FLANAGAN, David. JavaScript: O Guia Definitivo. 7. ed. Porto Alegre: Bookman, 2025. E-book. p.i. ISBN 9788582607008. Disponível em:
<https://integrada.minhabiblioteca.com.br/reader/books/9788582607008/>. Acesso em: 29 mai. 2026.
- ❖ MOREIRA, Sabrina B. Iniciando com JavaScript: um guia prático para entender os fundamentos da linguagem da web. 1. ed. [S.l.]: Kindle Direct Publishing, 2024. E-book Kindle. Disponível em:
<https://www.amazon.com.br/dp/B0CW9MLKTT>. Acesso em: 29 mai. 2026.

Referência da aula

- ❖ MILETTO, Evandro M.; BERTAGNOLLI, Silvia C. Desenvolvimento de software II: introdução ao desenvolvimento web com HTML, CSS, javascript e PHP. (Tekne). Porto Alegre: Bookman, 2014. E-book. p.119. ISBN 9788582601969. Disponível em: <https://integrada.minhabiblioteca.com.br/reader/books/9788582601969/>. Acesso em: 11 mai. 2026
- ❖ FREECODECAMP. Web page rendering on the browser - different methods. freeCodeCamp, 2022. Disponível em: <https://www.freecodecamp.org/news/web-page-rendering-on-the-browser-different-methods/>. Acesso em: 30 mar. 2026.

Referência da aula

- ❖ EDGEMESH. How to improve your website start render time. Edgemesh Blog, [s.d.]. Disponível em: <https://edgemesh.com/blog/how-to-improve-your-website-start-render-time>. Acesso em: 30 mar. 2026.
- ❖ RAMOTION. What is a CSS selector? Types, syntax and examples. Ramotion Blog, 2026. Disponível em: <https://www.ramotion.com/blog/css-selector/>. Acesso em: 30 mar. 2026.



univille.br